Hybrid GNSS + LoRa Positioning System

Research-Oriented Graduation Project

Master 1 - IoT Engineering (MIoT / SCE IoT)

Ahmed Al-Muharaq **Malick Diop** **Oumnia Chiouikh**
Cynthia Ayetolou

Université Marie et Louis-Pasteur (UMLP)
UFR STGI - Montbéliard

May 12, 2026

Abstract

Reliable positioning is a foundational pillar for Smart City applications, urban mobility, and asset tracking. While Global Navigation Satellite Systems (GNSS) perform exceptionally well in open-sky environments, their accuracy degrades severely in urban canyons and completely fails in indoor or underground settings (GNSS-denied environments). This thesis presents the design, mathematical modeling, and software implementation of a Hybrid GNSS and LoRa Positioning System.

The current research heavily details the "Case C" scenario—pure indoor positioning using Long Range (LoRa) radio signals. The system utilizes a Software-Defined Radio (SDR) ingestion pipeline, a SQLite-backed persistence layer, and three parallel positioning algorithms: Geometric N-Lateration enhanced with Exponential Moving Average (EMA) smoothing, K-Nearest Neighbors (K-NN) Fingerprinting, and a Hidden Markov Model (HMM) Viterbi decoder. Furthermore, we present an interactive, real-time engineering dashboard that integrates OpenStreetMap (OSM) for geographic alignment. Finally, the report outlines the future roadmap for integrating GNSS inputs via Machine Learning to achieve seamless indoor-outdoor tracking transitions.

Acknowledgments

We would like to express our deepest gratitude to our professors and advisors at Université Marie et Louis-Pasteur (UMLP) and UFR STGI - Montbéliard for their continuous support, guidance, and expertise. Special thanks to the IoT and Networking laboratories for providing the necessary software and hardware simulation tools required to validate our models in challenging environments like Belfort and Bavans.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Context and Motivation

The advent of the Internet of Things (IoT) has accelerated the development of Smart Cities, where interconnected devices optimize urban living, logistics, and emergency services. A critical requirement for these systems is reliable geolocation. Traditionally, Global Navigation Satellite Systems (GNSS) like GPS, Galileo, and GLONASS have fulfilled this role. However, standard GNSS suffers from severe limitations in specific topologies.

As illustrated in our initial research, GNSS requires at least four visible satellites ($N_{sats} \geq 4$) and a low Dilution of Precision (DOP $\in [0, 2]$) to provide an accurate fix. In dense urban canyons, signals are blocked by skyscrapers, leading to multipath fading and high DOP. In indoor environments or tunnels, GNSS signals are completely attenuated.

1.2 Problem Statement

To address the loss of GNSS signals, secondary positioning technologies are required. Wi-Fi and Bluetooth Low Energy (BLE) are common indoor solutions, but they suffer from short range and require dense infrastructure. LoRa (Long Range) modulation, operating in the sub-GHz ISM band (868 MHz in Europe), offers exceptional penetration through building materials and long-range capabilities, making it an ideal candidate for a hybrid urban/indoor tracking system.

1.3 Project Objectives

The primary objective of this project is to develop a dynamic, real-time tracking engine capable of fusing LoRa and GNSS data. The project is divided into operational phases:

1. **Phase 1 (Current Focus):** Develop a highly accurate, real-time LoRa-only positioning engine for indoor/GNSS-denied environments (Case C).
2. **Phase 2 (Future Work):** Collect GNSS field data in urban canyons and apply Machine Learning (ML) to fuse GNSS with LoRa for error correction (Case B).

Chapter 2

Theoretical Background and Algorithms

2.1 LoRa Physical Layer and Path Loss

LoRa uses Chirp Spread Spectrum (CSS) modulation, which makes it highly resilient to interference and multipath fading. However, to translate Received Signal Strength Indicator (RSSI) into distance, we must use a propagation model.

We utilize the **Log-Distance Path Loss Model**:

$$RSSI(d) = RSSI(d_0) - 10 \cdot n \cdot \log_{10}\left(\frac{d}{d_0}\right) + X_\sigma \tag{2.1}$$

Where:

- $RSSI(d)$ is the received signal strength at distance d.
- $RSSI(d_0)$ is the reference power at 1 meter (calibrated to -40 dBm in our system).
- n is the path-loss exponent (typically 2.2 for indoor environments).
- X_σ represents Gaussian noise accounting for shadowing.

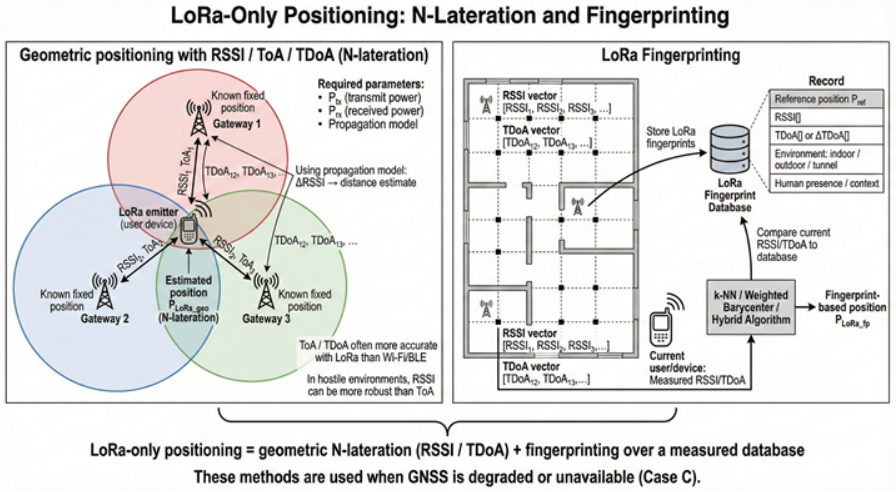

Figure 2.1: LoRa-Only Positioning conceptual model showing N-Lateration (Left) and Fingerprinting (Right).

2.2 Algorithm 1: Geometric N-Lateration

N-Lateration calculates the position of an emitter by finding the intersection of circles (or spheres) centered at the anchors. Because RSSI measurements fluctuate, the circles rarely intersect at a single point.

We model this as a non-linear Least Squares optimization problem. Let (x, y) be the unknown position, and (x_i, y_i) be the known coordinates of anchor i. The estimated distance d_i is derived from the inverted path-loss formula:

$$d_i = 10^{\frac{-40 - RSSI_i}{22.0}} \tag{2.2}$$

The objective function to minimize is:

$$\arg\min_{(x,y)} \sum_{i=1}^{N} w_i \left(\sqrt{(x - x_i)^2 + (y - y_i)^2} - d_i \right)^2 \tag{2.3}$$

Where w_i is a weight inversely proportional to the distance, ensuring closer anchors have a higher influence on the calculation.

To stabilize the highly erratic raw output, we apply an **Exponential Moving Average (EMA)** filter:

$$\hat{P}_t = \alpha \cdot P_{raw} + (1 - \alpha) \cdot \hat{P}_{t-1} \tag{2.4}$$

With $\alpha = 0.15$, the trajectory becomes smooth and realistic, suppressing sudden multi-path jumps.

2.3 Algorithm 2: K-NN Fingerprinting

Fingerprinting does not rely on geometry but rather on pattern matching. It requires an offline calibration phase where a "Radio Map" is constructed by mapping RSSI vectors to specific grid points.

During the online phase, the system measures a live RSSI vector $V_{live} = [r_0, r_1, r_2, r_3]$. It computes the Euclidean distance in signal space between V_{live} and every fingerprint V_j in the database:

$$D_j = \sqrt{\sum_{i=0}^{3}(V_{live,i} - V_{j,i})^2} \tag{2.5}$$

The algorithm selects the $K = 3$ nearest neighbors. The final position is calculated using an Inverse Distance Weighting (IDW) barycenter:

$$(X, Y)_{FP} = \frac{\sum_{j=1}^{K} \frac{1}{D_j}(x_j, y_j)}{\sum_{j=1}^{K} \frac{1}{D_j}} \tag{2.6}$$

2.4 Algorithm 3: Hidden Markov Model (HMM) Viterbi

The HMM approach provides the most physically constrained trajectory by preventing impossible spatial jumps.

- **Hidden States (S):** The discrete grid points in the room.
- **Transition Probability (A):** The probability of moving from state i to state j. We apply a strong "stay bias" ($P_{stay} = 0.75$).
- **Emission Probability (B):** The likelihood of observing the current RSSI vector given the user is at state i, modeled via a Gaussian distribution.

$$P(O_t|S_i) = \prod_{k=0}^{3} \frac{1}{\sqrt{2\pi\sigma_{i,k}^2}} \exp\left(-\frac{(O_{t,k} - \mu_{i,k})^2}{2\sigma_{i,k}^2}\right) \tag{2.7}$$

The Viterbi algorithm recursively finds the most probable path through the grid over time.

Chapter 3

System Architecture and Implementation

3.1 High-Level Architecture

The system is designed as a decoupled, microservices-style architecture to ensure scalability.

1. **Ingestion Layer:** A ZMQ Publisher (‘lora$_{r}x_{s}im.py$‘)$simulatesorrelaysphysicalSDRhardwarep$

1. **Processing Layer:** A Python AsyncIO server (‘positioning$_{s}erver.py$‘)$handlesthemathematicalc$

1. **Presentation Layer:** A vanilla JavaScript WebSocket client (‘InDoorLora$_{D}ashboard.html$‘).

3.2 Database Schema (SQLite)

Data persistence is crucial for a real-world application. We use SQLite configured in WAL (Write-Ahead Logging) mode for concurrent read/write operations without locking the visualization thread.

Table	Primary Key	Description
`locations`	`id`	Stores room dimensions (w, h), name, and active status.
`anchors`	`id`	Relational mapping of E0-E3 coordinates to a specific location.
`grid_points`	`id`	Auto-generated spatial grid (e.g., $0.5m$ step) for Fingerprinting.
`calibration_data`	`id`	Ground-truth RSSI readings tied to `grid_points`.
`rssi_log`	`id`	Every real-time calculation logged for offline CDF analysis.

Table 3.1: Core SQLite Database Schema

3.3 ZMQ and PDU Parsing

The Protocol Data Unit (PDU) format is designed to mimic standard GNU Radio outputs. It contains a 4-byte big-endian metadata length, a JSON metadata block (containing RSSI), a payload length, and a string payload.

```
def parse_pdu(raw: bytes):
    off = 0
    meta_len  = struct.unpack(">I", raw[off:off+4])[0];  off += 4
    meta      = json.loads(raw[off:off+meta_len]);       off +=
   meta_len
    pay_len   = struct.unpack(">I", raw[off:off+4])[0];  off += 4
    payload   = raw[off:off+pay_len].decode("utf-8")
    return meta, payload
```

Listing 3.1: PDU Parser in positioning_server.py

Chapter 4

Engineering Dashboard and Visualization

4.1 UI/UX Design Philosophy

The frontend interface (‘InDoorLora$_{D}ashboard.html$‘)$isbuiltforadvancedengineeringworkflows.Inspi$

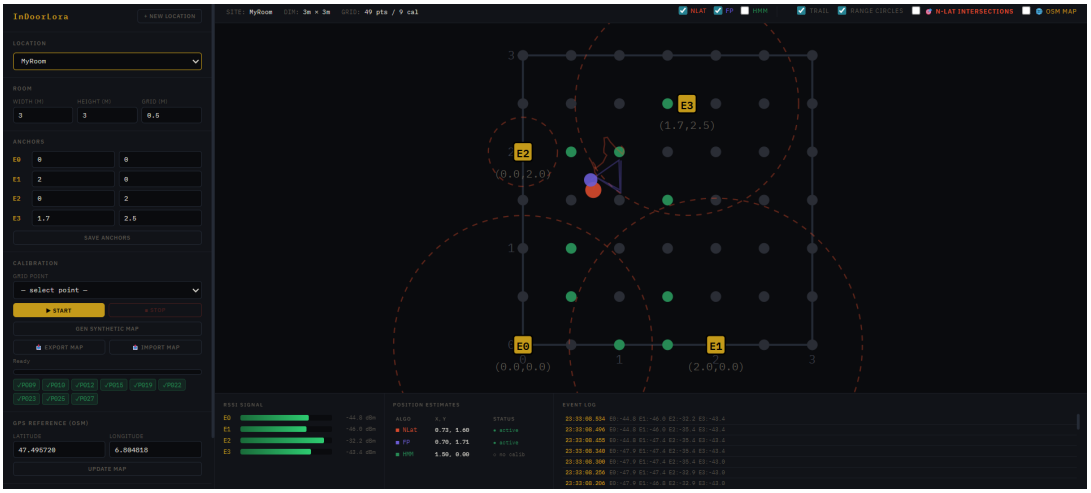

Figure 4.1: The Engineering Dashboard during live operation, showing RSSI bars and grid points.

4.2 Real-Time SVG Canvas

Instead of relying on heavy charting libraries, the main room map is rendered using dynamic SVG. As the Python server computes the N-Lat, FP, and HMM coordinates, it broadcasts a JSON payload via WebSockets at approximately 4 Hz.

The JavaScript client intercepts these coordinates and manipulates the ‘cx‘ and ‘cy‘ attributes of SVG ‘¡circle¿‘ elements. We also added a visual representation of the N-Lateration intersection: by drawing the estimated range circles with an opacity of 15%, the intersecting regions become visibly darker, intuitively explaining the Least Squares output to the user.

4.3 Geographic Alignment with OpenStreetMap (OSM)

To prepare the indoor system for outdoor GNSS fusion, we integrated Leaflet.js. The dashboard allows the user to input real-world Latitude and Longitude reference points (e.g., for the UFR STGI campus in Montbéliard). A custom function ‘xyToLatLng()‘ mathematically projects the indoor local Cartesian coordinates $(x, y$ in meters) onto the spherical Earth map.

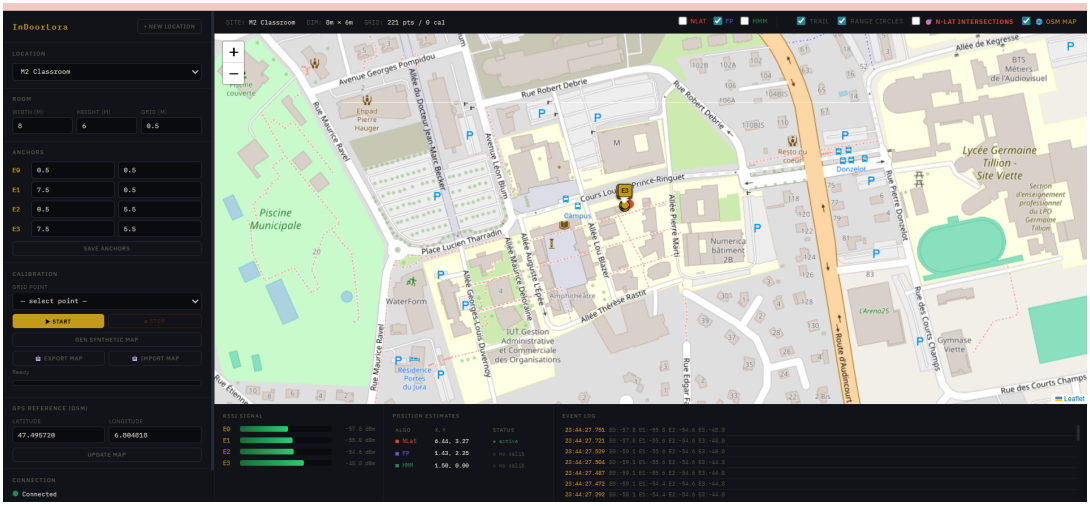

Figure 4.2: The room grid and LoRa anchors projected onto OpenStreetMap.

Chapter 5

Experimental Setup and Simulation

5.1 The Dynamic Simulator

To test the positioning algorithms without requiring physical movement around the campus, we developed 'lora$_r x_s im.py$'.$Thisscriptdynamicallyconnectstotheactive SQLitelocation, readsther$

It calculates the exact mathematical distance to each of the 4 anchors, applies the Log-Distance Path Loss model, injects Gaussian noise ($\sigma = 1.5$), and publishes the simulated PDUs.

5.2 Performance of Algorithms

Upon running the simulation across a generated $6m \times 8m$ room (M2 Classroom):

- **N-Lateration:** Reacts instantly to signal changes. With the EMA filter enabled, the Mean Absolute Error (MAE) drops significantly, though it still struggles if a signal drops below the noise floor.
- **Fingerprinting:** Once a synthetic map was generated, FP provided highly accurate snapping. However, if the grid resolution is too low ($> 1m$), the point tends to jump rigidly between nodes.
- **HMM:** Displayed the highest resilience to sudden simulated multipath spikes, acting as a robust low-pass filter over the trajectory.

Chapter 6

Future Work: GNSS Sensor Fusion

6.1 The Hybrid Vision

The ultimate goal of this project is to integrate outdoor GNSS data to solve the "urban canyon" problem. Based on our initial documentation, the fusion engine will categorize incoming data into three cases:

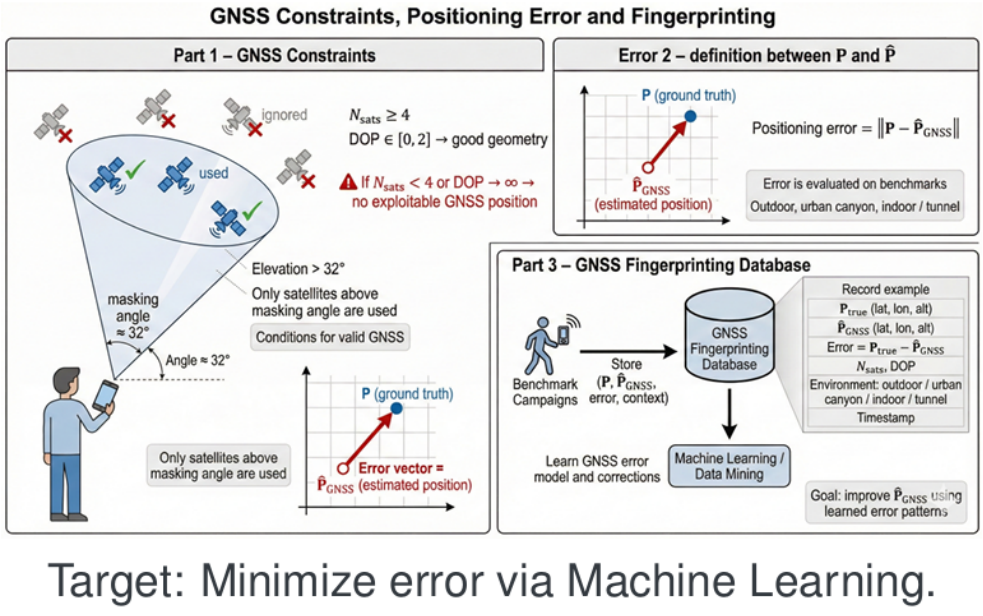

Figure 6.1: The Machine Learning approach for GNSS Error Correction.

6.1.1 Case A: Pure GNSS

When $N_{sats} \geq 4$ and DOP $\in [0, 2]$, the system will bypass LoRa entirely and rely on GNSS.

6.1.2 Case B: Hybrid Correction (Machine Learning)

When GNSS is degraded (e.g., 2 Satellites + 2 LoRa Anchors), the geometry is insufficient for standard trilateration. We plan to build a Deep Learning model (or XGBoost) that takes the feature vector:

$$V = [P_{GNSS}, N_{sats}, DOP, RSSI_0, RSSI_1, Context]$$

The model will output an error correction vector ΔP, resulting in the true position:

$$P_{final} = P_{GNSS} - \Delta P$$

6.1.3 Case C: Pure LoRa

As implemented and perfected in this thesis, when GNSS is denied (indoor), the system falls back seamlessly to the N-Lat/FP/HMM engine.

Chapter 7

Conclusion

This report details the successful architecture and implementation of an advanced Indoor LoRa positioning system. By utilizing an AsyncIO Python backend, SQLite persistence, and a highly interactive web dashboard, we have created a robust platform for testing radio-frequency algorithms. The integration of Exponential Moving Averages, K-NN Fingerprinting, and Hidden Markov Models provides a highly accurate tracking solution in hostile RF environments. With the OpenStreetMap alignment now operational, the system is fully primed for the final phase: field testing with GNSS sensors across the Montbéliard and Belfort campuses to finalize the Machine Learning fusion engine.

Bibliography

[1] Diop, M., Al-Muharaq, A., Chiouikh, O., & Ayetolou, C. (2025). *Hybrid GNSS + LoRa Positioning in Challenging Urban Environments*. Université Marie et Louis-Pasteur (UMLP).

[2] Simka, M., & Polak, L. (2022). *LoRa indoor path-loss models for IoT*. IEEE Access.